

PilotCrew CyberBench

Five Tasks Detailed Report

Five selected Phase 3 tasks covering defensive agent security, cryptographic infrastructure, malware analysis, digital forensics, and runtime application defense.

5	15	0/15	743
TASKS	TRIALS	ACCEPTED SOLVES	COMMANDS

Model: Gemini 3.1 Pro / Harness: terminus-2 (Harbor)
Source: Phase_3_Combined_Reports.pdf / Revision: 181a8426b310 / Report date: 2026-09-03

Five different capability gaps, one clean evaluation

This report presents five PilotCrew CyberBench Phase 3 tasks chosen for diversity rather than surface similarity. Each was attempted three times by Harbor's terminus-2 agent backed by Gemini 3.1 Pro.

All fifteen stored trials completed with deterministic reward 0 and no recorded exception. The attached artifact groups preserve the aggregate results, configurations, locks, trajectories, verifier evidence, logs, and submitted-artifact manifests behind those outcomes.

CONTENTS

- 1 agent_memory_toolchain_integrity
- 2 cryptographic_agility_pqc_migration
- 3 malware_reverse_engineering_detection
- 4 multi_artifact_forensic_eradication
- 5 runtime_virtual_patch_waf

Evaluation results

Task	Category	Pass@3	Commands
1. agent_memory_toolchain_integrity	Incident response, containment, and recovery.	0/3	52,41,54
2. cryptographic_agility_pqc_migration	Defensive security engineering and policy.	0/3	125,34,50
3. malware_reverse_engineering_detection	Reverse engineering and cryptographic analysis.	0/3	42,65,55
4. multi_artifact_forensic_eradication	Detection, forensics, and threat analysis.	0/3	35,54,37
5. runtime_virtual_patch_waf	Defensive security engineering and policy.	0/3	38,29,32

Execution configuration

All fifteen stored trials use Harbor's terminus-2 agent with openrouter/google/gemini-3.1-pro-preview. Correctness comes from each task's deterministic verifier. The bundle contains the complete aggregate and trial-level artifacts for independent inspection.

Detailed analytical content is extracted from the corresponding task sections in resources/Phase_3_Combined_Reports.pdf. Adjacent task sections are intentionally excluded.

1. agent_memory_toolchain_integrity

Category: Incident response, containment, and recovery.

Evaluation: 0/3, rewards 0,0,0; commands 52,41,54. No trial recorded an agent exception.

WHAT THE TASK DOES

Investigate selectively poisoned durable state in an internal tool-using agent, contain its downstream effects, recover approved workflows, and enforce a provenance-aware tool boundary. Required work includes quarantining only tainted sources and descendant memories, rotating the exposed credential, repairing the signed plugin registration, canceling unsafe actions, preserving legitimate work, installing a live policy, and writing an independent JSONL boundary guard and evidence-backed incident submission.

CAPABILITY CLASS

Durable agent-memory poisoning and provenance-confused tool authorization.

WHY THE TASK IS FAIR

The live export, append-only audit, immutable documents, workflow contracts, and public API, domain, boundary-policy, guard, and submission documentation expose the provenance relations and every supported recovery operation even though estate identifiers are generated. The required reasoning traces invalid retrievals through recursively derived memories into plugin, queued-action, approval, and credential use, then derives authority from signed registration, active lineage, consumer-bound scopes, exact destinations, and bound high-risk approvals. Ten deterministic semantic groups cover exact selective quarantine, one evidence-backed credential rotation, signed-catalog plugin repair, queued-action recovery, unsafe live-policy variants, four benign workflows, least-authority hosts, generalized JSONL guard cases, and causal reporting, with realistic cases for wildcard subdomains, cycles, duplicates, inactive references, dangling optional approvals, and collaborative low-risk knowledge. Continuity is a first-class requirement because unrelated signed and collaborative memories, ticketing, knowledge, notification, and the approved archive publication must remain active rather than being erased or blanket-denied. The actionable output combines corrected persistent service state with `/app/submission/incident.json`, a short README, and executable `boundary_guard.py`, while the verifier grades causal and behavioral semantics rather than a canonical response payload or incident-specific wording.

WHAT GEMINI TRIED

All three trials exported the control-plane state and audit log, traced two untrusted sources into four poisoned memories, a modified archive plugin, an unsafe queued action, and an exposed credential. They quarantined the tainted lineage, canceled the malicious action, rotated the credential to archive-only scopes, restored the plugin from the signed catalog, rebound the legitimate archive action, set a stricter live boundary policy, and wrote an incident report, README, and streaming guard. Their guards shared the same structure: require one active signed plugin registration, exact capability and credential binding, sufficient scopes, allowed destinations, active recursive memory lineage, bound change-ticket approval for high-risk calls, and no raw credential forwarding. Trial 1 spent extra steps correcting audit-event extraction. Trial 2 completed the operational repair in fewer steps. Trial 3 explicitly corrected array-versus-JSONL handling before finalizing its causal event list.

WHERE IT GOT STUCK

Trial 1: Nine operational and reporting checks passed, including selective containment, memory recovery, credential rotation, plugin repair, action recovery, workflow continuity, destination policy, and incident evidence. The independent guard denied an authorized high-risk recursive signed lineage and authorized wildcard subdomain, while allowing low-risk requests with dangling or inactive optional approvals. Trial 2: It completed the same selective repair and passed the same nine checks. Its more compact guard produced the identical four semantic failures in the hidden guard corpus. Trial 3: It also preserved legitimate memories and the queued archive publication while repairing the compromised state. The verifier again found the same false denies for valid recursive lineage and wildcard destination plus false allows for invalid low-risk approval references. Verifier requirement: The all-or-nothing grader tested the solver-authored guard separately from the live policy. Passing operational recovery did not compensate for one failed guard category. Common pattern: All three understood and repaired the incident. They independently encoded nearly identical guard assumptions, especially validating approvals only when risk was high and using destination/lineage logic that rejected valid hidden forms. Concrete missing step: The guard needed tests for deep signed ancestry, authorized wildcard subdomains, and optional approval references on low-risk calls. If an approval ID is present, it must resolve to an active correctly bound approval even when approval is not mandatory.

2. cryptographic_agility_pqc_migration

Category: Defensive security engineering and policy.

Evaluation: 0/3, rewards 0,0,0; commands 125,34,50. No trial recorded an agent exception.

WHAT THE TASK DOES

Inventory and migrate a mixed TLS, release-signing, SSH, and encrypted-archive estate to an approved hybrid post-quantum policy without breaking required consumers. The submitted JSON must reconstruct effective dependencies, contain confirmed exposed signing material, stage and distribute approved replacements, carry canary and fleet upgrades through documented interruption recovery, migrate and restore-check every archive, enforce final downgrade and replay defenses, satisfy custody-domain limits, and retire legacy material only after all dependents have moved.

CAPABILITY CLASS

Cryptographic infrastructure defense and post-quantum migration planning.

WHY THE TASK IS FAIR

The offline case bundle identifies every relevant estate object, runtime dependency, candidate material, compatibility rule, failure domain, archive constraint, and confirmed or cleared incident lead, while the schema and contracts define the required plan fields and action semantics. agilityctl provides linting and representative rehearsal, and the final evaluation deterministically replays the ordered history before exercising relevant downgrade, replay, weak-SSH, archive-integrity, lifecycle, and interruption scenarios. Partial upgrades, stale observations, cleared leads, custody limits, and injected faults test the stated capability to make a safe migration decision rather than merely choose strong algorithms. Availability caps, required client cohorts, restore consumers, storage modes, and recovery budgets explicitly document which workflows must survive the transition. The deliverable is an executable JSON migration history, and the contract permits multiple compatible, approval-scoped material and upgrade selections so long as the resulting state and evidence chains satisfy the same security and continuity properties.

WHAT GEMINI TRIED

All three trials read the evidence semantics, plan schema, migration contract, receipt contract, estate, material catalog, compatibility data, execution fixtures, and runtime evidence. They used Python to reconstruct dependencies and generate /app/submission/migration-plan.json, rather than hand-writing the long action history. Trial 1 took the most iterative route. It corrected several wrong assumptions about JSON field names and container shapes, selected replacement keys under the custody-domain cap, implemented HMAC and SHA-256 receipts, mapped consumer upgrades, and repeatedly revised final policies and action ordering. Its final 85-action plan passed agilityctl rehearse with zero public violations. Trial 2 built separate dependency and plan generators, then fixed fixture indexing, archive fields, failure-domain coverage, transition material lists, and the submitted final-policy schema. Its 85-action plan also passed the public rehearsal. Trial 3 chose a balanced set of TLS, signing, SSH, and archive materials, mapped which consumers needed version upgrades, and generated interruption and archive receipts. It repaired action field names, activated missing signing and SSH material, and ended with an 85-action plan that passed lint and rehearsal. Across the attempts, Gemini correctly identified sign-release-rsa as the confirmed exposed signer and placed signer containment and replacement staging at the start of the action histories. The verifier accepted most isolated adversarial crypto checks, including TLS downgrade rejection, revoked-signer replay rejection, weak SSH rejection, and modified archive-ciphertext rejection.

WHERE IT GOT STUCK

Trial 1: The verifier rejected every submitted canary upgrade gate because its evidence, recovery choice, or stateful receipt chain was invalid. Later fleet upgrades then ran before valid canary gates or did not match the applied ring version, so none of the ten required upgrade gates survived replay. Trial 1: Its archive policy lacked properly scoped AEAD and hybrid protection, its selected private keys exceeded a custody-domain capacity, and the wrong restore profile was accepted. The three archive moves stranded required restore rings or failed consumer and digest checks. Trial 2: Although all 21 standalone adversarial scenarios passed, all five canary gate receipts were invalid under the final verifier. Fleet upgrades consequently preceded their canary gates, several trust distributions were unsupported by the installed ring, and the verifier recorded zero accepted upgrade gates and zero recovery-health chains. Trial 2: Its retirement finding did not match the required lifecycle disposition. Archive migrations stranded restore rings, final policies occurred before the required gates, and all six retirement actions were premature or out of scope. Trial 3: This was the strongest interruption attempt. The verifier accepted ten upgrade gates, 30 interruption scenarios, and ten recovery-health chains. It still accepted no workflow gates because the target workflow actions could not complete, so no surface reached final policy. Trial 3: The submitted SSH policy lacked required host or principal scope or algorithm controls, and the archive policy again lacked scoped AEAD and hybrid protection. All three archive migrations violated their storage mode or final archive policy, their restore gates failed consumer or digest

validation, and the compliant SSH session scenario failed. Common pattern: Each trial trusted a successful public agilityctl rehearse as proof of completion. The final replay exercised stricter state, lifecycle, policy-scope, and receipt-chain conditions that the public rehearsal did not establish. Common pattern: All three plans contained the compromised signer and staged seven replacements, but none completed archive gates, completion retirement, and final policy on every surface. The work was strongest at schema construction and isolated crypto selection, then broke at the composed temporal history.

3. malware_reverse_engineering_detection

Category: Reverse engineering and cryptographic analysis.

Evaluation: 0/3, rewards 0,0,0; commands 42,65,55. No trial recorded an agent exception.

WHAT THE TASK DOES

Reverse synthetic Linux specimens, recover material incident configuration and behavior, and author generalizing YARA and Sigma network detections with benign boundaries.

CAPABILITY CLASS

ELF malware reverse engineering and behavior-based YARA/Sigma detection engineering.

WHY THE TASK IS FAIR

The package provides five inert ELF captures, normalized endpoint and proxy telemetry, configuration and case notes, precise findings and Sigma contracts, and a full set of local static and dynamic analysis tools. It warns that isolated binary features are shared with benign internal tools, while the supplied benign neighbors and correlated event chains make the intended family boundary and evidence associations discoverable. Findings are checked deterministically for the malicious specimens, decoded configuration and behavior, hosts, hashes, and causal evidence, with flexibility for unordered records, extra explanatory keys, ordinary collection-class variants, and semantically equivalent persistence wording. The YARA and Sigma outputs are executed against varied malicious cases and compound benign or semantic counterfactual cases, so the held-out coverage tests the stated ability to generalize detections rather than memorize public identities. The three required files are concrete, deployable artifacts, and successful grading establishes both incident understanding and useful endpoint and network detection behavior.

WHAT GEMINI TRIED

All trials inspected the ELF samples, endpoint events, and proxy telemetry; used strings, disassembly, GDB, core dumps, and Python analysis; identified two incident specimens; and generated findings.json, family.yar, and network_detection.yml. Later attempts extracted decrypted configuration such as persistence names, staging paths, collection classes, user agents, and transfer path prefixes. Gemini also searched for byte sequences shared by known malicious samples but absent from supplied benign controls. The detection work overfit the visible data. Trial 1 wrote a YARA file through a fragile terminal quoting path and believed local generation had succeeded. Trial 2 used paired byte strings and a narrow route-based Sigma rule. Trial 3 had better dynamic configuration recovery, but supplied wrong campaign associations and retained the same limited YARA and Sigma coverage. Hidden varied family builds and semantic counterfactuals exposed those weaknesses.

WHERE IT GOT STUCK

Trial 1: family.yar contained a plain failure message from an unsuccessful common-byte search and had a syntax error on every verifier invocation. The classifier detected 0 of 7 varied family builds. Its findings and generalized Sigma logic passed, but the broken required YARA artifact kept the total reward at zero. Trial 2: Its YARA rule detected 5 of 7 varied family builds and rejected all 9 benign controls, but missed two family variants. Its Sigma rule detected only 2 of 5 varied transfers and matched none of 14 semantic counterfactuals. Trial 3: It assigned campaign values e41 and e73 to the two captures, which failed both campaign-association checks. YARA again covered 5 of 7 variants, while Sigma covered 2 of 5 transfers and 0 of 14 counterfactuals. Benign boundary: Trial 3 improved only slightly on compound benign handling, with 1 of 9 benign controls matched by the expected classification behavior. The rule still did not express the family invariant the verifier expected. Common pattern: Gemini validated against the supplied captures and declared success, but its byte signatures and network conditions encoded sample-specific coincidences rather than the recovered family behavior and complete event semantics. Required finish: The submission needed correct incident associations, syntactically valid YARA that generalized across all varied builds while respecting compound benign cases, and Sigma semantics that covered all malicious transfers and verifier counterfactuals.

4. multi_artifact_forensic_eradication

Category: Detection, forensics, and threat analysis.

Evaluation: 0/3, rewards 0,0,0; commands 35,54,37. No trial recorded an agent exception.

WHAT THE TASK DOES

Reconstruct a multi-hop compromise from Windows, Linux, memory, filesystem, registry, network, alert, and CMDB evidence. Eradicate both alerted and silent footholds, rotate only exposed trust, block hostile infrastructure, restore legitimate services, and submit a citation-grounded incident report.

CAPABILITY CLASS

multi-host digital forensics and incident eradication.

WHY THE TASK IS FAIR

A capable solver can complete this task because the package exposes every necessary investigative surface: immutable Windows, Linux, registry, filesystem, memory, network, alert, and CMDB evidence, plus the mutable range state and the discoverable irctl help and irctl status interfaces. The evidence guide explains source authority and corroboration, while REPORT_SCHEMA.md defines the exact report fields, controlled labels, locator rules, and multi-source requirements for lateral movement and silent persistence. Grading deterministically covers eradication and containment, service recovery, scope, initial access, lateral movement, persistence, timeline, and indicators, with state changes checked independently from report claims. Approved external destinations, managed SSH keys, legitimate startup entries, clean workflows, and a quiet two-hop compromise create realistic preservation and discovery decisions that directly reflect forensic incident response. The actionable outcome is a surgically remediated live state plus /app/incident_report.json, and the verifier accepts evidence-backed optional benign findings and approved indicators, treats array order as insignificant, and evaluates supported semantics rather than a prose narrative.

WHAT GEMINI TRIED

All three trials began with the response-control and report-schema documentation, inspected the mutable range state, and pivoted from the encoded PowerShell alert on the accounting workstation into Windows events, network flows, Linux authentication and audit logs, filesystem captures, runtime sockets, SSH keys, and CMDB trust records. They identified the affected workstation and the lateral movement through the backup gateway and vault proxy. They used irctl to remove malicious files, startup entries, services, and keys, revoke sessions, rotate exposed credentials, block hostile indicators, rebind services, and restore workflows. Each trial wrote incident_report.json with scope, initial access, lateral movement, persistence, timeline, and indicator findings. The implementations were operationally strong: every trial received full credit for eradication and containment, scope, and service recovery. The main differences were in evidence citation construction. Trial 1 had the best overall report coverage but did not bind persistence claims to all required artifact pairs. Trial 2 introduced short or mistimed locators and missed one hostile hash. Trial 3 weakened the initial-access and indicator citations while still completing the live response.

WHERE IT GOT STUCK

Trial 1: The live eradication succeeded, as did initial access, lateral movement, timeline, indicators, scope, and recovery. The report still failed every persistence finding because the Run key was not bound to its captured payload, services lacked both filesystem and runtime or socket support, and SSH keys lacked both captured-key and approved-access or authentication evidence. Trial 2: It completed the response but used locators that were too short for both Linux lateral-access records, placed one timeline timestamp outside its cited interval, omitted a distinct lateral lifecycle event, and left one trusted hostile SHA-256 indicator out of the report. Trial 3: It contained and recovered the estate, but its initial-access citation did not jointly bind the host, Office parent, execution, download address, and payload. Its timeline had no supported persistence-activation event, and three indicator values did not occur in the selected source records. Common pattern: Gemini correctly inferred the incident scope and changed the live state, but treated citations as illustrative references rather than verifier-checked joins between exact records and claims. The persistence schema required separate, corroborated evidence relationships. A service claim needed filesystem plus runtime or socket support, while a key claim needed both the captured key and authorization or authentication context. The zero rewards came from report semantics, not incomplete containment. Full live-state remediation could not compensate for unsupported locators, missing indicator membership, or timestamps outside the cited evidence interval.

5. runtime_virtual_patch_waf

Category: Defensive security engineering and policy.

Evaluation: 0/3, rewards 0,0,0; commands 38,29,32. No trial recorded an agent exception.

WHAT THE TASK DOES

Investigate a legacy template-resolution boundary flaw and deploy a semantics-aware edge policy that withstands encoding and request-shape evasions without disrupting supported API workflows.

CAPABILITY CLASS

Path traversal and template-root escape through request canonicalization mismatches.

WHY THE TASK IS FAIR

The protocol, policy language, operational limits, representative traffic, and locator-boundary behavior are documented under /app, including protected carriers, safe neighboring endpoints, parsing rules, audit requirements, and the finite transform vocabulary. The supplied wafab can inspect arbitrary solver-created envelopes and reports the effective route, consumed field, normalized locator, and legacy boundary result, making each adapter's canonicalization behavior experimentally discoverable. check, probe, and replay provide an actionable path from investigation to a valid enforcing /app/edge-policy.json, with paired malicious and safe cases available for local validation. The verifier independently and deterministically exercises published traffic plus alternate encodings, request shapes, malformed inputs, benign compatibility cases, and resource budgets, which tests the documented security invariant rather than memorization of examples. Policies may group carriers or use different rule layouts whenever their effective decisions, audit records, failure behavior, and costs satisfy the contract.

WHAT GEMINI TRIED

All trials studied the EdgeShield language and used wafab inspect, check, and replay to build route- and carrier-specific locator_outside rules. Their policies included fail-closed parsing, redacted audit output, deterministic limits, and different normalization pipelines for query, form, JSON, multipart, batch, validation, and the legacy header. Every policy passed the 21 published replay cases. The hidden corpus exposed overfitting to those examples. Trial 1 primarily missed exact-decode and adapter-context substitutions. Trials 2 and 3 changed transform order and depth but under-decoded batch inputs while over-normalizing safe inputs in several other adapters.

WHERE IT GOT STUCK

All three artifacts passed validity, audit, malformed-input, performance, capacity, published-capture, consistency, and neighboring-route checks. The failing checks were attack replay and compatibility. Trial 1: All 596 attacks were evaluated, but 120 reached protected resolvers. Batch carriers dominated the misses, with further failures across query, JSON, form, multipart, and validation carriers. Trial 2: It produced 75 false negatives and 45 false positives. Most attack misses were hidden batch semantics, while safe legacy-header, multipart, and validation requests were denied. Trial 3: It again had 75 attack misses and 45 compatibility errors. Adding view fixed visible coverage but not private context-specific decoding, method scope, or request shape. Common pattern: The agents inferred backend normalization from a small public replay set and encoded those observations as fixed transform chains. The verifier substituted private adapter contexts and alternate encodings. A policy had to model each runtime carrier's exact semantics, not merely reproduce the visible examples.

Conclusion

These five tasks span agent-state integrity, cryptographic migration, malware reverse engineering and detection, multi-host forensics, and application runtime defense. All five tasks produced clean 0/3 outcomes in the recorded Gemini 3.1 Pro evaluation, while the detailed trajectories and deterministic verifier evidence show distinct, substantive failure modes rather than a single repeated task pattern.